

Introdução à Ciência da Computação com Python II

Recursão

REALIZAR APÓS Módulo 6 / Semana 6

Módulos 6 a 6 | 120 minutos | 100 pontos

Curso principal: Introdução à Ciência da Computação com Python Parte 2 |
<https://www.coursera.org/learn/ciencia-computacao-python-conceitos-2>

Conteúdo verificável: Recursão; decomposição; testes de casos base

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Nota de fonte: Módulo 7 é explicitamente extra (Scrapy e PyGame); excluído das provas obrigatórias.

Recursão e rastreamento

Defina $\text{soma}(n)=0$ se $n=0$, senão $n+\text{soma}(n-1)$, para $n \geq 0$. (a) Trace $\text{soma}(4)$. (b) Implemente. (c) Prove terminação pelo argumento que diminui e explique o que acontece fora do domínio.

Desenvolvimento / justificativa / esquema:

Potenciação recursiva

Implemente potência(a , n) para inteiro $n \geq 0$ usando divisão por 2. (a) Reutilize uma única chamada recursiva por nível. (b) Teste $n=0$, ímpar e par. (c) Compare número de multiplicações com a versão que reduz n em 1.

Desenvolvimento / justificativa / esquema:

Lista recursiva

Implemente máximo de lista não vazia por recursão em um índice, sem fatiar a lista. (a) Defina caso base. (b) Teste somente negativos. (c) Explique o custo e a importância de não usar 0 como máximo provisório.

Desenvolvimento / justificativa / esquema:

Recursão em strings

Implemente `palindromo(texto)` por recursão com índices. (a) Compare extremos e reduza o intervalo. (b) Defina os casos de string vazia e unitária. (c) Teste 'arara', 'abba', 'ab' e explique tempo e espaço.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Recursão e rastreamento (15 pontos)

Chamadas 4, 3, 2, 1, 0; retornos 0, 1, 3, 6, 10.

```
def soma(n):  
    if n == 0: return 0  
    return n + soma(n-1)
```

Inteiro não negativo diminui até 0. Com negativos, a recursão se afasta da base; contrato ou validação deve impedir o caso. Para entradas grandes a pilha real tem limite, embora a definição matemática termine.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Potenciação recursiva (25 pontos)

```
def potencia(a, n):  
    if n == 0: return 1  
    p = potencia(a, n//2)  
    if n % 2 == 0: return p*p  
    return p*p*a
```

Teste $2^0=1$, $2^5=32$, $2^6=64$. Há $O(\log n)$ níveis/multiplicações; versão linear usa $O(n)$. Guardar p evita calcular duas vezes o mesmo subproblema. Definição $a^0=1$ é a convenção do algoritmo.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Lista recursiva (25 pontos)

```
def maximo(a, i=0):  
    if i == len(a)-1: return a[i]  
    m = maximo(a, i+1)  
    return a[i] if a[i] > m else m
```

$O(n)$ tempo e $O(n)$ ativações; não há cópia por fatiamento. Para $[-4, -2, -9]$ retorna -2 . Inicializar com 0 criaria valor que nem está na lista e violaria o contrato em dados negativos. Lista vazia precisa de contrato ou erro explícito.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Recursão em strings (35 pontos)

```
def palindromo(texto, inicio=0, fim=None):  
    if fim is None: fim = len(texto)-1  
    if inicio >= fim: return True  
    if texto[inicio] != texto[fim]: return False  
    return palindromo(texto, inicio+1, fim-1)
```

Resultados True, True, False; vazio e unitária são True. Cada chamada reduz o intervalo em 2, garantindo terminação. Pior caso $O(n)$ tempo e $O(n)$ pilha; não cria cópias por fatias. Maiúsculas, espaços e acentos são comparados literalmente, conforme o contrato. A normalização não foi pedida.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.